1.00 Modeling Practices

↳ Its purposes are to understand (prioritized and designed) real-world system, visualize the current and desired system, document key decisions, especify a solution, plan construction and, above all, communicate efficiently.

↳ UML is a collection of modeling languajes: use case diagrams (users and goals), class diagrams (classes and relationships), interaction diagrams (how objects communicate) and state charts (internal behavior of objects).

→ Good practices:

1. Loosely coupled layers: User interface → Application logic → Persistance / Communications → Operating System
2. Prefer aggregation over inheritance: aggregation can change at runtime, so it's more flexible.
3. Program to an interface or abstraction: the reduces coupling and unnecessary dependencies
4. Meaningful identifiers: classes are nouns in PascalCase (Widget), variable use lowerCamelCase and reflect their purpose (newWidget), and methods are verbs (computeCost)
5. Unit tests: they help find design errors, verify correctness, document usage and enable regression testing. Use testing frameworks.

2.00 Introduction to UML

↳ is an object-oriented modeling language (really a collection of languajes) that is expressive, semi-formal, supports incremental development, is process independent, and is customizable and extensible

→ Building blocks: modeling elements are like words, diagrams like sentences or paragraphs, and models like whole papers or books.

→ Diagram types:

1. Structural: class, object, component, deployment.
2. Behavioral: use, case, sequence, collaboration (communication), state chart, activity.

→ Three modeling perspectives:

1. Analysis: real-world objects, the problem domain, the system as it is
2. Specifications: real-world and software objects, automation boundaries
3. Implementation: mostly sw objects, solution-oriented concepts.

3.00 Basic Use Cases

↳ Actors: things outside the system (users, other software, hardware) that have goals with respect to it.

Escaneado con CamScanner

↳ Use cases: what users need to accomplish, independent of design choices.

↳ Use cases vs. system interactions: use case answer what and why; interactions answer how.

↳ Use case diagram: actors, use cases, relationships and subjects (system boundary)

↳ Relationships between actors: generalization/specialization

↳ Relationships between use cases:
1. extend: variation of another use case under a condition, at an extension
2. include: one use case requires another's goal
3. generalization: one use case is a subset or refinement of another

↳ Finding actors: who or what uses the system, provides input, uses output, need support, or which other systems.

↳ Finding use cases: list actors and their activities, or list external events and the system's responses.

↳ Benefits: they capture user-visible functions, show who needs what, document reuse.

4.00 UML Quick Reference Guide (Analysis)

Keep an analysis perspective (model the real world, not software), let the incompleteness of diagram as a paragraph with one central idea.

→ Use case diagrams.
actor (noun, capitalized), use case (clear goal), a solid line without arrowheads between actor and use case include (break goals into sub-goals)

→ Class diagrams
class (noun), generalization, named binary association (use a meaningful verb phrase, not just "has") aggregation ("subpart of"), composition (the part is destroyed with the whole)

→ Activity diagrams
initial node, action, control flow, decision, fork/join, object flow, send/accept signal and final node

→ Interaction diagrams
communication diagrams shows objects, links and messages.

Escaneado con CamScanner

→ State chats
initial state, state, transition, internal activity, high level states, internal transitions, entry/exit actions and final state

Objects, Classes and Basic Class Diagrams

↳ Objects
are the basic building block of object-oriented systems

↳ usually not objects
events (sometimes), relationships between objects, behavior, and constraints

↳ Classes
Humans classify similar things by their observable properties, and classes are abstractions that let us handle whole collections of objects that share commonalities.

-Examples
Student, Course, Semester, Instructor, Department, College, Class section, Major, Grade

→ Three perspective

1. Analysis → A set of objects (the problem domain)
2. Specification → An interface to encapsulated software representation of objects
3. Implementation (Design → An abstract data type

↳ Class syntax and names

→ name, attributes, operations, responsibilities and user defined compartments

→ names should be singular nouns or noun phrases, meaning ful in the problem domain

→ Objects syntax

Similar to a class, but the name is instance Name: ClassName and u underline.

Escaneado con CamScanner

↳ Attributes

1. Analysis : an attribute relates an object to another object
2. Specification : an obligation to provide or manage one piece
3. Implementation : like data members in C#, C++ or Java
4. Syntax : Visibility is public +, protected # or private -. Name use lowerCamelCase.

Topic for exam
1. Clean code : rules of follow, standards, According to the programming language or the programming paradigm. Snake_case, CamelCase
2. Programming Paradigms : Structured programming (Procedural), OOP.
3. VCS : Git, we manage versioning of software development artifacts : code, test cases data files
4. OOP Concepts : Encapsulation, Abstraction, Inheritance, Polymorphism (Pillars of OOP)
objects and classes
- Encapsulation : classes, packages, libreries it is implemented using private
- Abstraction : abstract methods, interfaces, think in what really matters of my application domain, it is implemented using public, abstract, interfaces

Constructors use to initialize/instantiate objects of a class
Person person = new Person(1, "Edison Bascano", 4078.25f);
int a
a is not initialized
Person person; //can I use the person object? → no

Objects and classes : UML Class Diagram
Functionality and users : use Case Diagram
Classes : attributes and methos.

Classes relationships in Class Diagrams
- Dependencies : The function of one class uses object of another class, either as parameters or as return data types, or even inside the methos.
- Association : One object of class contains or has and attribute/object of another class
- Compositions : the life line of the composed objects depends on the life the composer object
- Aggregation : every object depends on itself to exist

Exceptions → handle users erros
setters → setting values to an attribute

Escaneado con CamScanner